American Computer Science League

2023 Finals ● Short Problems Solutions ● Senior Division

1. Boolean Algebra

$$\begin{aligned}\overline{A\overline{B}}(AB \oplus \overline{B}C) &= (\overline{A} + B)(AB\overline{\overline{B}C} + \overline{AB}\,\overline{B}C)\\ &= (\overline{A} + B)(AB(B + \overline{C}) + (\overline{A} + \overline{B})\,\overline{B}C)\\ &= (\overline{A} + B)(AB + AB\overline{C} + \overline{A}\,\overline{B}\,C + \overline{B}C)\\ &= (\overline{A} + B)(AB + \overline{B}C)\\ &= \overline{A}AB + \overline{A}\,\overline{B}\,C + AB + B\overline{B}C\\ &= \overline{A}\,\overline{B}\,C + AB\end{aligned}$$

A. $AB + \overline{A}\,\overline{B}\,C$

2. Boolean Algebra

$$\begin{aligned}\overline{A}(B \oplus \overline{C}) + B(\overline{A} + \overline{C}) &= \overline{A}(\overline{B}\,\overline{C} + BC) + B(\overline{A} + \overline{C})\\ &= \overline{A}\,\overline{B}\,\overline{C} + \overline{A}BC + \overline{A}B + B\overline{C}\\ &= \overline{A}\,\overline{B}\,\overline{C} + \overline{A}B + B\overline{C}\end{aligned}$$

A	B	C	$\overline{A}$	$\overline{B}$	$\overline{C}$	$\overline{A}\,\overline{B}\,\overline{C}$	$\overline{A}B$	$B\overline{C}$	$\overline{A}\,\overline{B}\,\overline{C} + \overline{A}B + B\overline{C}$
0	0	0	1	1	1	1	0	0	1
0	0	1	1	1	0	0	0	0	0
0	1	0	1	0	1	0	1	1	1
0	1	1	1	0	0	0	1	0	1
1	0	0	0	1	1	0	0	0	0
1	0	1	0	1	0	0	0	0	0
1	1	0	0	0	1	0	0	1	1
1	1	1	0	0	0	0	0	0	0

These 4 make it FALSE: (0, 0, 1), (1, 0, 0), (1, 0, 1), and (1, 1, 1)

C. 4

3. Bit-String Flicking

(LCIRC-2 1101101) OR (RSHIFT-1 1011101) XOR
1101101 AND 1110111
= 0110111 OR (0101110 XOR (1101101 AND 1110111))
= 0110111 OR 1001011 = 1111111

D. 1111111

4. Bit-String Flicking

Let X = abcde
LHS = (RCIRC-1 (LSHIFT-1 X)) OR NOT 10101 AND
(LCIRC-1 (RSHIFT-2 X))
= (RCIRC-1 (LSHIFT-1 abcde)) OR 01010 AND
= (LCIRC-1 (RSHIFT-2 abcde))
= (RCIRC-1 bcde0) OR 01010 AND (LCIRC-1 00abc)
= 0bcde OR (01010 AND 0abc0)
= 0bcde OR 0a0c0
= 0 b+a c d+c e
RHS = 01011
Therefore, b+a = 1, c = 0, d+c = 1 e = 1.
Therefore, d = 1. a+b = 1 so (11011, 10011, 01011) are 3 bit strings.

B. 3

5. Recursive Functions

$f(1) = 0$
$f(2) = 1$
$f(3) = 2*f(3-1) + f(3-2) = 2*f(2) + f(1) = 2*1 + 0 = 2$
$f(4) = 2*f(4-1) + f(4-2) = 2*f(3) + f(2) = 2*2 + 1 = 5$
$f(5) = 2*f(5-1) + f(5-2) = 2*f(4) + f(3) = 2*5 + 2 = 12$
$f(6) = 2*f(6-1) + f(6-2) = 2*f(5) + f(4) = 2*12 + 5 = 29$
$f(7) = 2*f(7-1) + f(7-2) = 2*f(6) + f(5) = 2*29 + 12 = 70$
$f(8) = 2*f(8-1) + f(8-2) = 2*f(7) + f(6) = 2*70 + 29 = 169$
$f(9) = 2*f(9-1) + f(9-2) = 2*f(8) + f(7) = 2*169 + 70 = 408$
so the 9th term is greater than 256. This generates Pell numbers.

B. 9

6. Recursive Functions

$f(54) = f([(54-4)/2]) - 1 = f(25) - 1 = 65 - 1 = 64$
$f(25) = f([(25-4)/2]) - 1 = f(10) - 1 = 66 - 1 = 65$
$f(10) = f(2 * 10 + 1) + 3 = f(21) + 3 = 63 + 3 = 66$
$f(21) = f([(21-4)/2]) - 1 = f(8) - 1 = 64 - 1 = 63$
$f(8) = 8^2 = 64$
$f(64) = f([(64-4)/2]) - 1 = f(30) - 1 = 84 - 1 = 83$
$f(30) = f([(30-4)/2]) - 1 = f(13) - 1 = 85 - 1 = 84$
$f(13) = f(2 * 13 + 1) + 3 = f(27) + 3 = 82 + 3 = 85$
$f(27) = f([(27-4)/2]) - 1 = f(11) - 1 = 83 - 1 = 82$
$f(11) = f(2 * 11 + 1) + 3 = f(23) + 3 = 80 + 3 = 83$
$f(23) = f([(23-4)/2]) - 1 = f(9) - 1 = 81 - 1 = 80$
$f(9) = 9^2 = 81$
Therefore, $f(f(54)) = f(64) = 83$

B. 83

7. Digital Electronics

The Boolean expression for the circuit is: $\overline{(AB)(B \oplus C)} + C$

$\overline{(AB)(B \oplus C)} + C = \overline{AB} + \overline{B \oplus C} + C$
$= \overline{A} + \overline{B} + (B \oplus \overline{C}) + C$
$= \overline{A} + \overline{B} + \overline{B}\,\overline{C} + BC + C$
$= \overline{A} + (\overline{B} + \overline{B}\,\overline{C}) + (BC + C)$
$= \overline{A} + \overline{B}(1 + \overline{C}) + C(B + 1)$
$= \overline{A} + \overline{B} + C$

D. $\overline{A} + \overline{B} + C$

8. Digital Electronics

The Boolean expression that represents the digital circuit is:

$\overline{(A + (AB + BC))\,\overline{C}} = \overline{(A + (AB + BC))}\,\overline{\overline{C}}$
$= \overline{(A + BC)}\,\overline{\overline{C}}$
$= \overline{A\overline{C} + BC\overline{C}}$
$= \overline{A\overline{C}}$
$= \overline{A} + C$

If $\overline{A} + C = 0$, then $\overline{A} = 0$ and $C = 0$.
So $A = 1$ and $B =$ * and $C = 0$. (1, *, 0) makes it FALSE.

B. 2

9. Prefix-Infix-Postfix

6 4 $ 2 5 $ * 2 2 ^ − 2 4 ^ $
= ((6 $ 4) * (2 $ 5) − (2 ^ 2)) $ (2 ^ 4)
= ([$\sqrt{|24|}$] * [$\sqrt{|10|}$] − 4) $ 16
= (4 * 3 − 4) $ 16
= [$\sqrt{|8 * 16|}$] = [$\sqrt{|128|}$] = 11

B. 11

10. Prefix-Infix-Postfix

$(x - h)^2 / a^2 + (y - k)^2 / b^2 = 1$ written in prefix is:
(^ − x h 2) / (^ a 2) + (^ − y k 2) / (^ b 2) = 1
(/ ^ − x h 2 ^ a 2) + (/ ^ − y k 2^ b 2) = 1
(+ / ^ − x h 2 ^ a 2 / ^ − y k 2^ b 2) = 1
= + / ^ − x h 2 ^ a 2 / ^ − y k 2^ b 2 1

D. =+/^−xh2^a2/^−yk2^b21

11. Computer Number Systems

One method to create the octal values of the Fibonacci sequence is to add the two previous octal numbers to get the next one. Thus, starting with 1, 1, 2, 3, 5 which are the same in both base 10 and base 8, the next one is $8_{10}=10_8$. $5_8+10_8=15_8$, $10_8+15_8=25_8$, $15_8+25_8=42_8$, $25_8+42_8=67_8$, $42_8+67_8=131_8$, $67_8+131_8=220_8$.
Another way is to convert the 12th term of the Fibonacci sequence into octal: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144 = 220_8.

C. 220

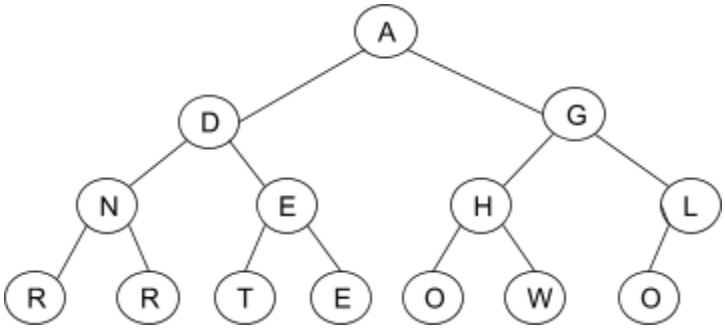
12. Computer Number Systems

The binary equivalents are: 100 = 1100100, 101 = 1100101, 102 = 1100110, 103 = 1100111, 104 = 1101000, 105 = 1101001, 106 = 1101010, 107 = 1101011, 108 = 1101100, 109 = 1101101, 110 = 1101110, 111 = 1101111. From 100-103, only 1 number has 101, from 104-111, all of them do which is 8. 106, 107, and 109 all contain 2 different 101 substrings. Therefore, 101 occurs 12 times.

D. 12

13. Data Structures

A heap is stored in an array and visualized by a binary tree filled in from top to bottom, left to right. Letters are filtered up a single branch of the tree when they are less than their parent.
The final tree is:



The nodes at depth 2 are: NEHL

D. NEHL

14. Data Structures

Following the algorithm as stated, these stack operations are done from left to right, then top to bottom:

PUSH(3), PUSH(2), x=POP(), y=POP(), PUSH(9), PUSH(5), x=POP(), y=POP(), PUSH(4), PUSH(4), PUSH(8), x=POP(), y=POP(), PUSH(12), x=POP(), y=POP(), PUSH(48), PUSH(6), x=POP(), y=POP(), PUSH(8)
Therefore, all of the values on the stack are: 3,2,9,5,4,4,8,12,4,48,6,8. The last item on the stack is the value of the postfix expression.

A. 3 2 9 5 4 4 8 12 48 6 8

15. Graph Theory

To find all of the paths of length 4, take the original adjacency matrix to the 4th power as follows:

$$\begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 3 & 0 & 1 & 2 & 1 \\ 2 & 1 & 1 & 3 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix}^2 = \begin{bmatrix} 9 & 2 & 5 & 8 & 5 \\ 8 & 3 & 5 & 9 & 5 \\ 2 & 3 & 3 & 5 & 3 \\ 0 & 0 & 0 & 0 & 0 \\ 2 & 3 & 3 & 5 & 3 \end{bmatrix}$$

The sum of the numbers in each row is 29+30+16+16 = 59+32 = 91.

D. 91

16. Graph Theory

The cycles ABA, ADA, AEA, BCB, and DED have a length of 2. There are 7 others: ACBA, ADEA, ABEA, ABEDA, ACBEA, ACBEDA, AEDA.

C. 7

17. What Does This Program Do?

This program finds the number of swaps needed to put the list in sorted order. The two outside loops find the location of each number, 0 to 9, in the current array. The inside loop swaps the values in the locations between where the number should be and where it currently is and counts how many swaps are made. The following table shows the resulting array and the swaps made:

x	array	count
original	8 6 2 9 3 5 7 1 4 0	0
0	0 4 1 7 5 3 9 2 6 8	5
1	0 1 4 7 5 3 9 2 6 8	6
2	0 1 2 9 3 5 7 4 6 8	9
3	0 1 2 3 9 5 7 4 6 8	10
4	0 1 2 3 4 7 5 9 6 8	12
5	0 1 2 3 4 5 7 9 6 8	13
6	0 1 2 3 4 5 6 9 7 8	14
7	0 1 2 3 4 5 6 7 9 8	15
8	0 1 2 3 4 5 6 7 8 9	16

C. 16

18. LISP

CONS makes X the first item in Y
Therefore, Z = ((a b (a c) (a (b c))) (c b) (a b c))
REVERSE reverses the order of each item, not the nested ones.
Therefore, U = ((a b c) (c b) (a b (a c) (a (b c))))
Both V and W get the third item in the list.
V = (CADDR U) = (CAR (CDR (CDR U))) = (a b (a c) (a (b c)))
W = (CADDR V) = (CAR (CDR (CDR V))) = (a c)

A. (a c)

19. FSA and Regular Expressions

The regular expression is:

[^e] [a-hlr-y]* [\s]* [a-n]* [^mr]* [\S]* [0-9]*

a) english - starts with an e
b) trig - satisfies
c) python 8 - satisfies
d) french 2 - satisfies
e) java#8 - satisfies
f) c++ - satisfies
g) biology - satisfies
h) adv physics - satisfies

There are 7 of them.

D. 7

20. Assembly Language

This program finds the maximum prime factor of a given number. The outside loop repeatedly divides the number by the prime factor that was found until the value of NUM is less than 2. Thus, 440/2 = 220/2 = 110/2 = 55/5 = 11/11 = 1. The inside loop finds each factor that goes in evenly starting with 2 each time. It checks 2, 3, 4, 5, …, but only prime numbers are found because others have been divided out already. Once a factor is found, it is compared to the current value of M which is changed if CNT > M so the largest value is printed which is 11.

D. 11